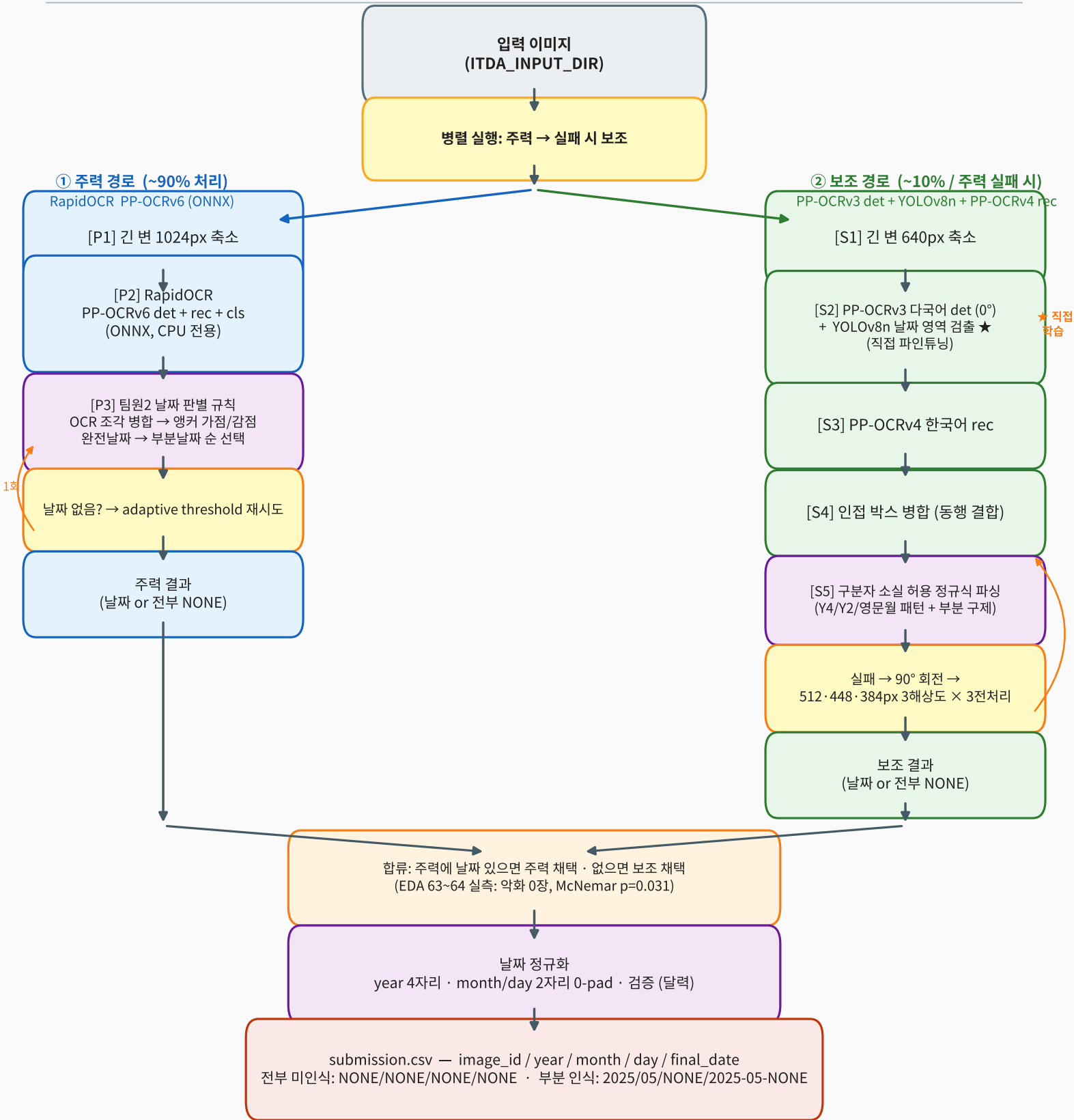


[DScover_카피바라] 소비기한 추출 파이프라인 아키텍처

제3회 ITDA 연합학술제 | 채점 환경: 4-Core CPU, Python 3.10, 오프라인



검증 결과 요약 (홀드아웃2 198장 — 설계에 쓰지 않은 표본)

구성	부분점수	완전일치	장당	500장 환산
보조 경로 단독 (PP-OCrv3/v4 + YOLO)	66.8%	59.1%	2.19초	1,096초
주력 경로 단독 (RapidOCR PP-OCrv6)	79.8%	75.8%	0.37초	184초
둘을 합침 (제출 구성)	82.3%	77.8%	0.53초	267초

타임아웃 2,400초 기준 → 267초 사용 (11%) | 여유 2,133초

1. 기술 선택의 근거 (실측 기반)

- RapidOCR(PP-OCrv6 ONNX)를 주력 엔진으로 선택한 이유
 - 기존 PP-OCrv3/v4 대비 홀드아웃2에서 +13.0%p (개선 49장 / 악화 14장, McNemar $p < 0.0001$)
 - onnxruntime 기반 → PyTorch 불필요. torch 임포트만으로 파이프라인 2.4배 느려짐(EDA 52)
 - 장당 0.37초(기존 2.19초) — 정확도·속도 동시 개선, 모델 세대 차이에 기인
- YOLOv8n 날짜 영역 검출기 직접 파인튜닝 (가산점 해당)
 - 배포 이미지 3,352장 중 날짜 영역 직접 라벨링해 학습(11MB ONNX 경량 모델)
 - PP-OCrv3 det가 놓치는 잉크젯·저대비 영역을 보충 — 보조 경로 recall 향상
- 복합 파이프라인 합류 전략 선택 이유 (A안 채택)
 - A(RapidOCR 주·실패시 보조): 82.3%, 악화 0장, $p=0.031$
 - B(RapidOCR 주·NONE 칸만 보조): 82.2%, 악화 3장 — 필드 혼합 오류 발생
 - C(기존 주·실패시 RapidOCR): 78.6%, D(다수결): 69.2% — 열등

2. CPU 환경 속도 최적화 전략

- PyTorch/Ultralytics 런타임 완전 제거
 - requirements.txt에 torch·ultralytics 없음. YOLOv8n도 ONNX로 변환하여 onnxruntime으로 구동
 - OMP 충돌 제거: paddle + torch 공존 시 OpenMP 스레드 경합 → 2.4배 속도 저하(EDA 53)
- 해상도 및 재시도 전략 실측 최적화
 - 주력 1024px: 3200px까지 올려도 recall 불변, 비용만 1.7→2.8초 (EDA 5)
 - 보조 640px: 960px보다 빠르고 정확 (홀드아웃 실측)
 - 회전 합집합($0^{\circ}+90^{\circ}$) 판별기 없이: 판별기 +0.2%p뿐, 합집합 +11.6%p (EDA 4)
- 박스 병합으로 비용 0에 정확도 대폭 향상
 - 날짜 한 줄이 2개 박스로 쪼개짐(20221 + 104 등) → 병합 후 정확도 48%→68% (EDA 19)
- 전처리 단일 적용 미채택
 - 13종 전처리 실험, $n=400$ 에서 +3.0%p지만 McNemar $p=0.201$ (유의미하지 않음)
 - 속도 비용 대비 효과 불충분. 주력 경로 실패분에만 adaptive threshold 재시도 적용

3. 직접 라벨링 및 검증 체계 (가산점)

- 총 383장 직접 판독·검수 (배포 데이터 정답 없는 환경에서 자체 구축)
 - 튜닝셋 137장: 보조 경로 파라미터 조정에 사용
 - 홀드아웃1 48장 / 홀드아웃2 198장: 설계에 쓰지 않은 독립 검증셋
 - 라벨링 도구(HTML+Python) 직접 제작 → 기존·근거 custom_data/README.md 명시
- 실측 기반 설계 결정 — 가설이 아닌 데이터로 검증
 - 기각 목록: 7자리 느슨 패턴(coverage +12%p, 정확도 0), 전처리 13종($p=0.201$),
 - RapidOCR 단독(vs 합친 구성 대비 악화 없이 +2.5%p 확보)

4. 도메인 활용 및 확장 계획

- 즉시 배포 가능 구조: Git clone → pip install → download_weights.sh → 추론
 - 오프라인 전용 설계(인터넷 없음) — 물류창고·현장 IoT 기기에 그대로 탑재 가능
 - 단일 프로세스 0.53초/장 — 카메라 1대당 초당 약 2장 처리 (실시간 라인 적용 가능)
- 활용 시나리오
 - ① 편의점·마트 자동 유통기한 관리: 진열 스캔 → 기한 임박 상품 자동 알림
 - ② 식품 기부 푸드뱅크: 기증품 소비기한 자동 분류 → 폐기 최소화
 - ③ 물류 입출고 자동화: 박스/캔류 날짜 인식 → ERP 연동 재고 수명 추적
- 확장 방향
 - YOLOv8n 파인튜닝 데이터셋 확대 → det recall 개선으로 NONE 비율 추가 감소
 - RapidOCR 모델 업그레이드(PP-OCrv6 → 차기 버전) — 아키텍처 수정 없이 교체 가능

5. 알려진 한계 및 미대응 케이스

- 잉크젯 각인 저대비·곡면 날짜: CLAHE 보정에도 인식을 낮음 (OCR 출력 자체가 없음)
- 연도 없이 월·일만 있는 경우(10.14): 무게·가격 오탐 우려로 미대응
- DATE_RE_COMPACT(구분자 없는 8자리): 바코드 번호 오탐 가능 — 키워드 게이트 추가 고려

성능 수치의 출처: EDA 스크립트(eda/01~67) 실측, 홀드아웃 설계에 사용하지 않은 독립 표본

홀드아웃2(198장)는 보조 경로 파라미터 조정에 여러 차례 쓰여 낙관 편향 가능성 있음 — 단, 최종 채택한 RapidOCR 주력 경로는 이 표본에 한 번도 노출되지 않음

직접 학습 YOLOv8n 가중치 (date_v1.onnx, 11MB): Release Assets 배포, download_weights.sh로 취득